

CSEN/CSIS402: Computer Organization and Systems programming
Spring Term 2026

Project milestone 2- Deadline: May 18, 2026

You need to modify your basic computer developed in milestone 1 by:

Optional: A version of milestone 1 solution is uploaded to the CMS in case you weren't able to make it work during milestone 1. Use this version, if you want, to achieve milestone 2. Some edits may still be done by you.

Adding the control unit (decoders, sequence counter and logic gates) necessary to implement the following program only.

No need for it to implement all the instructions of the basic computer. Make sure that all unused control signals are equal to zero to ensure running a stable program.

Remember you need to control all the loads, bus signals, ALU signals and so on for each Time T_x .

NOTES:

1. The IR register should NOT be a counter.
2. The PC register can be implemented using a counter instead of a regular register.
3. The available registers do not have clear or increment so you must re-think the timing diagrams of each instruction. However, you can design your registers to have clear and increment.
4. The instruction will be divided as follows:
 - Bits 12-15: opcode part
 - Bits 4-11: Address Field
 - The rest of the bits are not used and can be ignored in this project.
5. No need to handle indirect memory reference instructions. You can assume that all memory instructions are direct ones.
6. All instructions should start execution at T_3 , as no indirect addressing in this project.

Program	Assembly
<pre> int A, B, C, i; // variables for(i=0; i<3; i++) { A += A XOR B; C += A / B; } //Final Output: A =101 and C =35 </pre>	<pre> ORG 0 LOP, LDA A XOR B ADD A STA A LDA A DIV B ADD C STA C ISZ i BUN LOP HLT A, DEC 15 B, DEC 5 C, DEC 0 i, DEC -3 </pre>

The same instructions used in the lectures are used here, with the addition of two new instructions, DIV and XOR, which perform division and logical XOR operations between memory values and the accumulator.

Examples:

DIV 22 means $AC \leftarrow AC / \text{MEM}[22]$

XOR 55 means $AC \leftarrow AC \text{ XOR } \text{MEM}[55]$

Use the opcodes of BSA and AND for DIV and XOR as they are not implemented or needed in this project.

The list of instructions you need to work on:

LDA, STA, ISZ, BUN, ADD, DIV, XOR

You can ignore the HLT instruction.

You will need to develop the control and timings for each instruction above. You might also need to add some hardware in addition to the ones you know in the lecture (decoders, counters...).

→ Try with different values of A, B, and C to test your program.
For example: A = 15, B = 5, and C = 0, the final values are A = 101 and C = 35.

N.B.

Also ignore the interrupt. Just use the normal instruction cycle in everything.

Deliverables:

1) The circuit file in Logisim

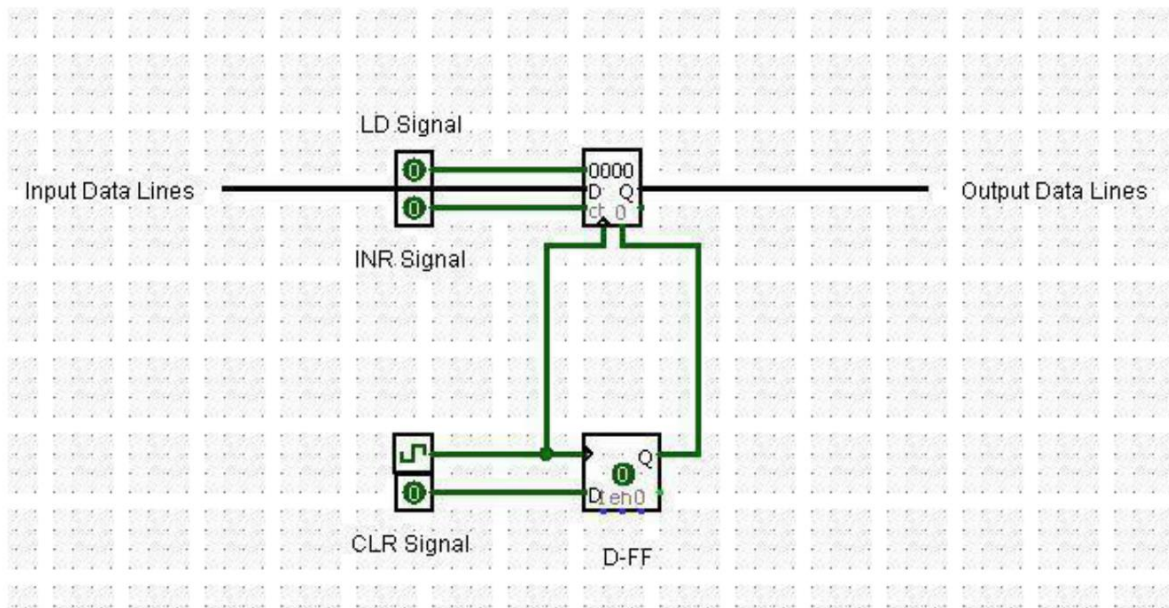
2) A small report containing each instruction you used with its timing (T0,T1,...) and control signals circuits that you developed for each register and component.

Due date and delivery and evaluation:

- Submission by May 18 11:59 pm.
- Submission link: <https://forms.gle/54VMrGNG4rhZ9C14A>
- Submit One Zipped folder containing the following:
 - Circuit file
 - Report file:
 - ✓ It should contain the RTLs for each instruction in the list of instructions starting T3.
 - ✓ You should include RTLs from T0-T2 once for all instructions.
 - ✓ You should include the logical expressions for each LD, INC and CLR of all registers and the sequence counter.
 - ✓ You should include the logical expressions for the read and write of the memory.
 - ✓ You should include all logical expressions of the encoder inputs of both the BUS select lines and the ALU select lines
 - Memory contents as text file.
 - A text file containing your names, ID, tutorial number and emails.
- Evaluation: During revision week.

Extra notes

The clear signal for counter registers in Logisim is asynchronous, which means that once the signal becomes 1, the register clears without waiting for the next positive clock transition. To solve this, add a flip flop to the CLR signals of only the registers you will use. This flip flop will enforce synchronous signal receipt as it is only activated at the edge. (This connection is shown in the diagram below)



1. Use Tunnel from Wiring components to prevent having a lot of interconnected wires. Note that tunnels **DO NOT** work across circuits. It **MUST** be used within the same circuit.
2. Edit the memory components by adding the machine code of the program after conversion.
3. To prevent clearing the memory contents each time you close the circuit file, save the contents as a .txt file by a right click on the memory.
4. Next time you open the circuit, right click on the circuit and load the pre-saved text file.